Foreign Key constraints

- Suppose we have the following schema :

Students(cuid: *string*, name: *string*, gpa: *float*)

Enrolled(student_id: *string*, cid: *string*, grade: *string*)

- And we want to impose the following constraint:

Only bona fide students may enroll in courses' i.e. a student must appear in the Students table to enroll in a class

Students

cuid	name	gpa
102	Bob	3.9
123	Mary	3.8

Enrolled

student_id	cid	grade
102	CS1	A
123	CS4	A+

We say that cuid is a foreign key that refers to Students

Declaring Foreign Keys

Students(cuid: *string,* name: *string,* gpa: *float*)
Enrolled(student_id: *string*, cid: *string,* grade: *string*)

```
CREATE TABLE Enrolled (
  student_id CHAR(20),
  cid  CHAR(20),
  grade  CHAR(10),
  PRIMARY KEY (student_id, cid),
  FOREIGN KEY (student_id) REFERENCES Students(cuid)
)
```

Foreign Keys and update operations

Students(cuid: *string*, name: *string*, gpa: *float*)

Enrolled(student_id: *string*, cid: *string*, grade: *string*)

- What if we insert a tuple into Enrolled, but no corresponding student?

  INSERT is rejected (foreign keys are constraints)!

- What if we delete a student?
  1. Disallow the delete
  2. Remove all of the courses for that student
  3. *SQL allows a third via NULL*

DBA chooses

Keys and Foreign Keys

Company

CName	StockPrice	Country
GizmoWorks	25	USA
Canon	65	Japan
Hitachi	15	Japan

What is a foreign key vs. a key here?

Product

PName	Price	Category	Manufacturer
Gizmo	$19.99	Gadgets	GizmoWorks
Powergizmo	$29.99	Gadgets	GizmoWorks
SingleTouch	$149.99	Photography	Canon
MultiTouch	$203.99	Household	Hitachi

shlomi-noach on Dec 9, 2016 · edited by shlomi-noach Edits Contributor •••

At GitHub we do not use foreign keys, ever, anywhere.

Personally, it took me quite a few years to make up my mind about whether foreign keys are good or evil, and for the past 3 years I'm in the unchanging strong opinion that foreign keys should not be used. Main reasons are:

- FKs are in your way to shard your database. Your app is accustomed to rely on FK to maintain integrity, instead of doing it on its own. It may even rely on FK to cascade deletes (shudder).
  When eventually you want to shard or extract data out, you need to change & test the app to an unknown extent.
- FKs are a performance impact. The fact they require indexes is likely fine, since those indexes are needed anyhow. But the lookup made for each `insert` / `delete` is an overhead.
- FKs don't work well with online schema migrations.

This last bullet is not a chicken and an egg, as you might think. FKs impose a lot of constraints on what's possible and what's not possible.

Here's an old post of mine, reviewing the first appearance of Facebook's OSC, and which includes some thoughts on foreign keys: http://code.openark.org/blog/mysql/mk-schema-change-check-out-ideas-from-oak-online-alter-table

Let's say you have two tables, P & C, standing for Parent & Child, respectively. There's a foreign key in C such that each row in C points to some "parent" value in P.

Doing schema migration of C is possible. However since foreign keys have unique names, the new (migrated) C table will have a FK with a different name than the original one.

Doing schema migration of P is just not going to work. Recall that `gh-ost` renames the table at the end. Alas, when renaming a table away, the FK will move with the renamed table. To create a parent-side FK on the *ghost* table, one would need to migrate C ; and because `gh-ost` uses async approach, P and P-ghost are never in complete sync at any point in time (except at lock time) which makes it impossible for C to have both a FK to P and to P-ghost. some integrity will be broken.

There's more discussion on the documentation of pt-online-schema-change

https://github.com/github/gh-ost/issues/331

☺ 👍 290 👎 60 🎉 8 🙂 22 ❤️ 60 🚀 11 👀 46